our shielding · Your smart contracts, our shielding · Your smart c

shieldify

SpringX

SECURITY REVIEW

Date: 7 April 2026

CONTENTS

1. About Shieldify **3**

2. Disclaimer **3**

3. About SpringX **3**

4. Risk classification **3**

4.1 Impact 3

4.2 Likelihood 4

5. Security Review Summary **4**

5.1 Protocol Summary 4

5.2 Scope 4

6. Findings Summary **4**

7. Findings **5**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, for which this report is intended, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About SpringX

SpringX is a modular staking + yield system built around a fixed-APR farm model with multi-reward support and externalized asset custody.

At a high level, the main functionalities are:

- Users stake assets into pools
- Rewards accrue deterministically based on fixed annual rates (APR)
- Assets are isolated in vaults and optionally deployed into strategies

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 4 days with a total of 64 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one Medium and eight Low severity issues. They're mainly related to edge-case accounting inconsistencies, unsafe integrations with strategies/tokens, and missing safeguards in upgradeability and fund flow handling.

The SpringX team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	SpringX
Repository	TheOfficialSpringX
Type of Project	Staking & Yield Farming Protocol (Fixed-APR)
Security Review Timeline	4 days
Review Commit Hash	83b86d28af5264e6046eed8552a4a8ea9f36dc8d
Fixes Review Commit Hash	7cecbfc8918f88b935354724a1b15f78aaff8468

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
vaults/SpringXVault.sol	151
farm/SpringXPointFixedApyFarm.sol	425
Total	**576**

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 1
- **Low** issues: 8
- **Info** issues: 6

ID	Title	Severity	Status
[M-01]	Raw `IERC20.approve()` in `setVaultStrategy()` is Incompatible with USDT0	Medium	Fixed
[L-01]	Missing `_disableInitializers()` in `SpringXVault` Implementation	Low	Fixed
[L-02]	Strategy Detach Without Fund Recall Can Strand User Deposits	Low	Fixed
[L-03]	Zero-Stake Users Cannot Claim Previously Accrued Underfunded Rewards Without Restaking	Low	Fixed
[L-04]	Non-Mintable Reward Transfer Revert Can Block Normal Exit	Low	Fixed
[L-05]	Unsafe Strategy Replacement Can Lead to Fund Locking	Low	Fixed
[L-06]	Incorrect Handling of Native Mode in `vaultBalance()`	Low	Fixed
[L-07]	Unaccounted Native Transfers Inflate TVL via Vault–Farm Integration	Low	Fixed
[L-08]	TVL Includes Yield While Reward Accounting Uses Deposits Only	Low	Fixed
[I-01]	Old Strategy Approval Not Revoked on Strategy Replacement	Info	Fixed
[I-02]	Mutable `setAssets()`, `setCoreAddress()`, and `setMainChef()` Can Break Vault Routing and Lock User Operations	Info	Fixed
[I-03]	`SpringXVault` Missing Storage Gap (`__gap`)	Info	Fixed
[I-04]	`_syncUserArrays()` Comment vs Behavior Mismatch (Documentation Issue)	Info	Fixed
[I-05]	`poolUsers` `EnumerableSet` Grows Monotonically	Info	Fixed
[I-06]	Deposit/Withdraw Fee-on-Transfer Handling Inconsistency	Info	Acknowledged

7. Findings

[M-01] Raw `IERC20.approve()` in `setVaultStrategy()` is Incompatible with USDT0

Severity

Medium Risk

Description

The `SpringXVault.setVaultStrategy()` directly calls `IERC20(assets).approve(...)` twice — once to reset to zero and once to approve `type(uint256).max` for the new strategy. The protocol confirms USDT0 is one of the supported assets for `SpringXVault`. USDT0 is built on the Tether `TetherTokenV2`/OFT-extension implementation, which historically does not return a boolean value from `approve()`.

Solidity's generated ABI decoder for the `IERC20.approve()` interface expects the callee to return a `bool`. When the call returns with empty return data (no bool), the decoder reverts. This means `setVaultStrategy()` will **always revert** for any vault whose asset is USDT0, permanently bricking strategy migration for those vaults.

Note that `TransferTokenHelper.safeTokenApprove()` — a low-level helper already present in the codebase that handles non-bool-returning tokens correctly — is imported by `SpringXVault` but is not used at this call site.

Location of Affected Code

File: vaults/SpringXVault.sol#L116-L134

```
function setVaultStrategy(IStrategy _strategy) external onlyOwner {
    strategy = _strategy;

    if (address(_strategy) != address(0)) {
        if (address(assets) != nativeAddress) {
            // Both calls use raw IERC20.approve — reverts if token
                returns no bool (e.g. USDT0)
            IERC20(assets).approve(address(_strategy), 0);
                                    // Line 128
            IERC20(assets).approve(address(_strategy), type(uint256).max)
                ;     // Line 129
            transferERC20ToStrategy();
        } else {
            transferNativeToStrategy();
        }
    }
}
```

File: common/TransferTokenHelper.sol#L35-L38

```
function safeTokenApprove(address token, address to, uint value) internal
    {
    (bool success, bytes memory data) = token.call(abi.encodeCall(IERC20.
        approve, (to, value)));
    require(success && (data.length == 0 || abi.decode(data, (bool))), '
        ...');
}
```

Impact

For any `SpringXVault` whose `assets` are USDT0, calling `setVaultStrategy()` will revert unconditionally. The owner cannot:

- Attach an initial strategy.
- Replace an underperforming or compromised strategy.
- Detach-and-re-attach a strategy for emergency migration.

Strategy migration is fully DoS'd for USDT0 vaults, and funds held in the vault remain inaccessible to any strategy permanently.

Proof of Concept

1. Deploy `SpringXVault` with `assets = USDT0`.
2. Owner calls `setVaultStrategy(strategyAddress)`.
3. Execution reaches `IERC20(assets).approve(address(_strategy), 0)`.
4. USDT0's `approve()` executes but returns no data.
5. Solidity's ABI decoder attempts to read a `bool` from empty return data reverts.
6. `setVaultStrategy()` is permanently uncallable for this vault.

Recommendation

Replace the two raw `IERC20.approve()` calls with `TransferTokenHelper.safeTokenApprove()` in `SpringXVault.setVaultStrategy()`:

```
-IERC20(assets).approve(address(_strategy), 0);
-IERC20(assets).approve(address(_strategy), type(uint256).max);

+TransferTokenHelper.safeTokenApprove(address(assets), address(_strategy)
    , 0);
+TransferTokenHelper.safeTokenApprove(address(assets), address(_strategy)
    , type(uint256).max);
```

Additionally, harden `safeTokenApprove()` itself to enforce the zero-first reset internally so no caller can skip it:

```
 function safeTokenApprove(address token, address to, uint value)
     internal {
+     if (value != 0) {
+         (bool s, bytes memory d) = token.call(abi.encodeCall(IERC20.
    approve, (to, 0)));
+         require(s && (d.length == 0 || abi.decode(d, (bool))), '
    TransferTokenHelper -> safeTokenApprove: Reset FAILED');
+     }
      (bool success, bytes memory data) = token.call(abi.encodeCall(IERC20
          .approve, (to, value)));
      require(success && (data.length == 0 || abi.decode(data, (bool))), '
          TransferTokenHelper -> safeTokenApprove: Approve Token FAILED');
```

```
}
```

Alternatively, use OpenZeppelin's `SafeERC20.forceApprove()`, which is already used in `SpringXPointFixedAPYFarm` and handles both the zero-first reset and non-bool-returning tokens.

Team Response

Fixed.

[L-01] Missing `_disableInitializers()` in `SpringXVault` Implementation

Severity

Low Risk

Description

The `SpringXVault` uses an upgradeable pattern (`Initializable`) with an external `initialize()`, but the implementation is not locked with `_disableInitializers()`. If left uninitialized, anyone can initialize the implementation and become its owner (`__Ownable_init(msg.sender)`).

Location of Affected Code

File: vaults/SpringXVault.sol#L93-L104

```
contract SpringXVault is IVault, Initializable, OwnableUpgradeable,
    ReentrancyGuardUpgradeable {
    function initialize(
        IERC20 _assets,
        address _nativeAddress,
        address _mainChef
    ) public initializer {
        __Ownable_init(msg.sender);
        __ReentrancyGuard_init();
        assets = _assets;
        nativeAddress = _nativeAddress;
        mainChef = _mainChef;
    }
    // no constructor calling _disableInitializers()
}
```

Impact

No direct proxy takeover in normal deployments, but the implementation instance can be taken over. This creates implementation-level risk (misconfiguration, abuse and potential loss of funds accidentally sent to the implementation).

Proof of Concept

1. Get the `SpringXVault` implementation address.
2. Call `initialize(...)` on the implementation.
3. Attacker becomes the owner of the implementation instance and can call the owner-only setters.

Recommendation

Add a constructor to lock the implementation:

```
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
    _disableInitializers();
}
```

Keep initialize as the proxy initialization entrypoint.

Team Response

Fixed.

[L-02] Strategy Detach Without Fund Recall Can Strand User Deposits

Severity

Low Risk

Description

In `setVaultStrategy()`, the vault updates `strategy` before recalling funds from the previously configured strategy.

When the owner sets `strategy` to `address(0)`, the branch that transfers funds to a strategy is skipped, and no recall from the old strategy occurs. Funds already deployed to the old strategy remain outside the vault withdrawal path.

This is owner-triggered and therefore primarily an admin-misconfiguration risk. Even so, if this mistake happens in production, users can lose access to funds that remain stuck in the detached strategy path.

Location of Affected Code

File: vaults/SpringXVault.sol#L116-L134

```
function setVaultStrategy(IStrategy _strategy) external onlyOwner {
    strategy = _strategy;

    if (address(_strategy) != address(0)) {
        if (address(assets) != nativeAddress) {
            IERC20(assets).approve(address(_strategy), 0);
```

```
            IERC20(assets).approve(address(_strategy), type(uint256).max)
                ;
            transferERC20ToStrategy();
        } else {
            transferNativeToStrategy();
        }
    }

    emit SetVaultStrategy(address(_strategy));
}
```

Impact

If assets are currently deployed in an existing strategy, detaching with `address(0)` can strand those assets in the old strategy.

Users may then fail to withdraw expected amounts because vault withdrawals follow only the currently configured strategy (or local vault balance), not the detached one. In practice, an admin mistake can translate into direct user loss of access to the principal.

Proof of Concept

1. User deposits assets into the vault while strategy A is active.
2. Strategy A receives and holds deployed vault assets.
3. Owner calls `setVaultStrategy(address(0))`.
4. Vault strategy pointer is cleared without recalling assets from strategy A.
5. User attempts withdrawal through vault/farm flow.
6. Withdrawal path no longer targets strategy A, and funds held there are effectively stranded from normal protocol flow.

Recommendation

Recall funds and revoke allowance from the old strategy before replacing or detaching it.

```
function setVaultStrategy(IStrategy _strategy) external onlyOwner {
    IStrategy oldStrategy = strategy;

    if (address(oldStrategy) != address(0)) {
        if (address(assets) != nativeAddress) {
            uint256 oldBal = oldStrategy.balanceOf();
            if (oldBal > 0) {
                oldStrategy.withdraw(address(this), oldBal);
            }
            IERC20(assets).approve(address(oldStrategy), 0);
        }
    }

    strategy = _strategy;

    if (address(_strategy) != address(0)) {
        if (address(assets) != nativeAddress) {
            IERC20(assets).approve(address(_strategy), 0);
```

```
            IERC20(assets).approve(address(_strategy), type(uint256).max)
                ;
            transferERC20ToStrategy();
        } else {
            transferNativeToStrategy();
        }
    }

    emit SetVaultStrategy(address(_strategy));
}
```

Team Response

Fixed.

[L-03] Zero-Stake Users Cannot Claim Previously Accrued Under-funded Rewards Without Restaking

Severity

Low Risk

Description

The farm records unpaid non-mintable rewards in `userAccrued` when reward liquidity is temporarily insufficient, but the public claim path is still gated by active stake.

When a user fully withdraws, `withdraw()` settles rewards, reduces `user.amount`, and then calls `_distributeRewards()`. If the reward token is underfunded at that time, the unpaid amount remains in `userAccrued`.

Later, after the operator tops up reward tokens, the same user cannot call `harvest()` because `harvest()` reverts when `user.amount == 0`. There is no standalone claim function for already-accrued rewards, so users must restake before they can claim.

Location of Affected Code

File: farm/SpringXPointFixedApyFarm.sol

```
function withdraw(uint256 poolId_, uint256 amount_) external nonReentrant
    whenNotPaused {
    // code
    _settleUser(poolId_, msg.sender);

    user.amount -= amount_;
    pool.totalStaked -= amount_;
    _updateAllRewardDebts(poolId_, msg.sender);

    // Unpaid non-mintable rewards may remain in userAccrued
    _distributeRewards(poolId_, msg.sender);
```

```
    // code
}

function harvest(uint256 poolId_) external nonReentrant whenNotPaused {
    // code
    UserInfo storage user = userInfos[poolId_][msg.sender];
    if (user.amount == 0) revert NoStake();
    // code
}

function _distributeRewards(uint256 poolId_, address user_) internal {
    // code
    if (!rewards[i].mintable) {
        uint256 farmBalance = IERC20(rewards[i].token).balanceOf(address(
            this));
        if (farmBalance >= amount) {
            // pay
        }
        // Insufficient balance: keep in accrued, no revert
    }
}
```

Impact

Users who fully exit while a non-mintable reward is underfunded can end up with non-zero accrued rewards that are not directly claimable.

Proof of Concept

1. User stakes in a pool that includes a non-mintable reward token.
2. User accrues rewards over time.
3. Farm reward token balance becomes insufficient.
4. User calls full `withdraw()`.
5. During `_distributeRewards()`, underfunded non-mintable rewards remain in `userAccrued`.
6. Operator later tops up the reward token balance.
7. User calls `harvest()` and receives `NoStake()` because `user.amount == 0`.
8. User must restake (non-zero), call `harvest()`, then optionally withdraw again to recover accrued rewards.

Recommendation

Allow claiming already-accrued rewards independently of active stake.

Option A (minimal change): permit `harvest()` for zero-stake users when `userAccrued` is non-zero.

```
function harvest(uint256 poolId_) external nonReentrant whenNotPaused {
    if (!pools[poolId_].exists) revert PoolNotExist();

    UserInfo storage user = userInfos[poolId_][msg.sender];
    _updatePool(poolId_);
    _syncUserArrays(poolId_, msg.sender);
```

```
    if (user.amount > 0) {
        _settleUser(poolId_, msg.sender);
        _updateAllRewardDebts(poolId_, msg.sender);
    }

    uint256[] storage accrued = userAccrued[poolId_][msg.sender];
    bool hasReward;
    for (uint256 i = 0; i < accrued.length;) {
        if (accrued[i] > 0) { hasReward = true; break; }
        unchecked { ++i; }
    }
    if (!hasReward) revert NoReward();

    _distributeRewards(poolId_, msg.sender);
}
```

Option B: add a dedicated `claimAccrued(poolId_)` function that only distributes existing accrued balances and does not require stake.

Team Response

Fixed.

[L-04] Non-Mintable Reward Transfer Revert Can Block Normal Exit

Severity

Low Risk

Description

For non-mintable rewards, the farm distributes rewards using a direct `safeTransfer()` call. If the configured reward token can block transfers for specific users (blacklist) or globally (pause), that transfer can revert.

The protocol has stated they intend to use USDT0 as a non-mintable reward token, and USDT0 includes blacklist controls. This makes the failure mode deployment-relevant rather than hypothetical.

Because reward distribution is coupled into normal user flows, the revert can block:

- `withdraw()` (called before principal withdrawal)
- `harvest()`
- `deposit()` for users who already have stake (auto-harvest path)

The protocol still provides `emergencyWithdraw()` as a principal rescue path, so this is not a permanent principal lock. However, users can be forced to use the emergency exit and forfeit accrued rewards.

Location of Affected Code

File: farm/SpringXPointFixedApyFarm.sol

```
function withdraw(uint256 poolId_, uint256 amount_) external nonReentrant
    whenNotPaused {
    // code
    _distributeRewards(poolId_, msg.sender);
    pool.vault.withdrawTokenFromVault(msg.sender, amount_);
}

function harvest(uint256 poolId_) external nonReentrant whenNotPaused {
    // code
    _distributeRewards(poolId_, msg.sender);
}

function deposit(uint256 poolId_, uint256 amount_) external payable
   nonReentrant whenNotPaused {
    // code
    if (user.amount > 0) {
        _settleUser(poolId_, msg.sender);
        _updateAllRewardDebts(poolId_, msg.sender);
        _distributeRewards(poolId_, msg.sender);
    }
}

function _distributeRewards(uint256 poolId_, address user_) internal {
    // code
    if (!rewards[i].mintable) {
        uint256 farmBalance = IERC20(rewards[i].token).balanceOf(address(
            this));
        if (farmBalance >= amount) {
            accrued[i] = 0;
            claimed[i] += amount;
            IERC20(rewards[i].token).safeTransfer(user_, amount);
        }
    }
}
```

Impact

When the non-mintable reward token reverts on transfer, affected users can lose normal liveness for reward-coupled actions and may be forced into `emergencyWithdraw()`.

- Principal is still recoverable through emergency exit.
- Accrued rewards are forfeited in an emergency exit.
- Impact is operational and economic (reward loss), not a full permanent fund lock.

Proof of Concept

1. Configure a pool with a non-mintable blacklisting reward token (USDT0, per protocol statement).

2. User stakes and accrues non-mintable rewards.
3. Reward token admin blacklists that user (or pauses transfers globally).
4. User calls `withdraw()`.
5. `_distributeRewards()` reaches `safeTransfer` for the non-mintable reward and reverts.
6. The entire `withdraw()` transaction reverts before the principal transfer.
7. User calls `emergencyWithdraw()` to recover principal (assuming stake-token transfer path itself remains functional), but accrued rewards are forfeited.

Recommendation

Decouple reward transfer failures from principal operations:

```
if (!rewards[i].mintable) {
    uint256 farmBalance = IERC20(rewards[i].token).balanceOf(address(this
        ));
    if (farmBalance >= amount) {
        try IERC20(rewards[i].token).safetransfer(user_, amount) {
            accrued[i] = 0;
            claimed[i] += amount;
            emit Harvested(poolId_, user_, rewards[i].token, amount);
        } catch {
            // keep accrued unchanged so user can retry later
        }
    }
}
```

Team Response

Fixed.

[L-05] Unsafe Strategy Replacement Can Lead to Fund Locking

Severity

Low Risk

Description

The `setVaultStrategy()` function replaces the existing strategy without handling the previous one. When a new strategy is set, the contract directly transfers available funds to it but does not withdraw funds already deposited in the old strategy or revoke its approvals.

This leads to a state where funds may remain in the old strategy while new funds are moved to the new one, without any built-in migration or recovery mechanism.

Location of Affected Code

File: vaults/SpringXVault.sol#L116-L134

```
function setVaultStrategy(IStrategy _strategy) external onlyOwner {
    strategy = _strategy;

    if (address(_strategy) != address(0)) {
        if (address(assets) != nativeAddress) {
            IERC20(assets).approve(address(_strategy), 0);
            IERC20(assets).approve(address(_strategy), type(uint256).max)
                ;
            transferERC20ToStrategy();
        } else {
            transferNativeToStrategy();
        }
    }

    emit SetVaultStrategy(address(_strategy));
}
```

Impact

Funds deposited in the previous strategy may become inaccessible, resulting in fragmented balances and potential permanent fund locking.

Proof of Concept

A user deposits funds while StrategyA is active. The owner later updates the strategy to StrategyB. The vault transfers its current balance to StrategyB, but funds already held in StrategyA remain there with no mechanism to retrieve them.

Recommendation

Ensure safe migration before updating the strategy by withdrawing all funds from the old strategy and clearing approvals, then assigning the new strategy.

Team Response

Fixed.

[L-06] Incorrect Handling of Native Mode in `vaultBalance()`

Severity

Low Risk

Description

The `vaultBalance()` function always calls `assets.balanceOf(address(this))` without handling the native currency case. However, in native mode `(address(assets) == nativeAddress)`, assets is not a valid ERC20 token, and calling `balanceOf()` will revert.

This creates an inconsistency with the `balance()` function, which correctly handles both ERC20 and native modes.

Location of Affected Code

File: vaults/SpringXVault.sol#L168-L170

```
function vaultBalance() external view returns (uint256) {
    return assets.balanceOf(address(this));
}
```

Impact

In native mode, calls to `vaultBalance()` will revert, breaking integrations or frontends relying on this function for liquidity checks.

Proof of Concept

Set the vault to native mode where `address(assets) == nativeAddress`. Calling `vaultBalance()` attempts to execute balanceOf on a non-ERC20 address, causing a revert.

Recommendation

Handle native mode consistently:

```
if (address(assets) == nativeAddress) {
    return address(this).balance;
}
return assets.balanceOf(address(this));
```

Team Response

Fixed.

[L-07] Unaccounted Native Transfers Inflate TVL via Vault–Farm Integration

Severity

Low Risk

Description

The `Vault` contract allows arbitrary native currency transfers via `receive()`, and native ETH can also be forcibly sent to the contract via `selfdestruct` (similarly, arbitrary ERC20 tokens can be transferred directly). These unsolicited transfers are not tracked in `totalAssets`. However, the Farm contract relies on `vault.balance()` to compute the pool's Total Value Locked (TVL) through the `getPoolTVL()` function.

Since `vault.balance()` includes `address(this).balance`, any unsolicited ETH sent directly to the Vault is reflected in the TVL calculations without being tied to actual user deposits. This creates a discrepancy between the gross assets held and the tracked user principal, breaking the implicit invariant that TVL represents only real user-supplied assets.

Location of Affected Code

File: farm/SpringXPointFixedApyFarm.sol

```
// Vault
receive() external payable {}

// Farm
function getPoolTVL(uint256 poolId_) public view returns (uint256) {
    return pools[poolId_].vault.balance();
}
```

Impact

An attacker or user can artificially inflate the reported pool TVL by sending ETH directly to the Vault. Because preventing forced native or token transfers is not possible on-chain, this ambiguity can easily mislead integrations, analytics dashboards, or any dependent logic that relies on `getPoolTVL()` under the assumption that it strictly represents user deposits.

Proof of Concept

1. User deposits assets normally into the Vault.
2. Attacker sends ETH directly to the Vault using `receive()` (or force-sends via `selfdestruct`).
3. `vault.balance()` increases due to address(this).balance.
4. `getPoolTVL()` returns an inflated TVL that is not strictly backed by user deposits.

Recommendation

Since restricting direct ETH or ERC20 transfers is ineffective against mechanisms like selfdestruct, the protocol should clarify the accounting through explicit naming and separated view functions:

1. Rename the existing view to reflect that it returns gross managed assets rather than tracked user principal.
2. Expose a separate, principal-only view such as `getPoolPrincipal()` or `getPoolDeposits()` that returns only the tracked user deposits (e.g., `pool.totalStaked()` or `vault.totalAssets()`, depending on the preferred vault-side terminology).

Team Response

Fixed.

[L-08] TVL Includes Yield While Reward Accounting Uses Deposits Only

Severity

Low Risk

Description

The `getPoolTVL()` function retrieves TVL using `vault.balance()`, which includes both user deposits and any yield generated by the strategy. In contrast, reward calculations in the Farm rely on `pool.totalStaked`, which only tracks user-deposited principal.

This creates a semantic mismatch where TVL reflects a higher value (including yield), while reward distribution is based solely on deposited amounts. Additionally, any direct token or native transfers to the Vault further inflate TVL without affecting reward accounting.

Location of Affected Code

File: farm/SpringXPointFixedApyFarm.sol#L525-L528

```
function getPoolTVL(uint256 poolId_) public view returns (uint256) {
    return pools[poolId_].vault.balance();
}
```

Impact

TVL reported to frontends and integrations may be inflated compared to the actual value used for reward calculations, potentially misleading users or external systems.

Recommendation

Clarify the distinction in documentation or provide a separate function (e.g., `getPoolDeposits()`) that returns `pool.totalStaked` for accurate deposit-based metrics.

Team Response

Fixed.

[I-01] Old Strategy Approval Not Revoked on Strategy Replacement

Severity

Informational Risk

Description

In `setVaultStrategy()`, the vault grants approval to the new strategy but does not revoke allowance from the previous strategy before replacing the strategy pointer.

Under a trusted-admin/trusted-strategy model, this is primarily a hardening issue (QA) rather than a direct high-severity exploit. However, stale approvals are still unsafe operationally and should be cleaned up during migration.

Location of Affected Code

File: vaults/SpringXVault.sol#L116-L134

```
function setVaultStrategy(IStrategy _strategy) external onlyOwner {
    strategy = _strategy;

    if (address(_strategy) != address(0)) {
        if (address(assets) != nativeAddress) {
            IERC20(assets).approve(address(_strategy), 0);
            IERC20(assets).approve(address(_strategy), type(uint256).max)
                ;
        }
    }
}
```

Impact

Residual allowance can remain on a retired strategy address. If old operational components are later compromised, the stale approval can become an unnecessary risk surface.

Recommendation

Revoke old strategy approval before updating `strategy`:

```
function setVaultStrategy(IStrategy _strategy) external onlyOwner {
    if (address(strategy) != address(0) && address(assets) !=
       nativeAddress) {
        IERC20(assets).approve(address(strategy), 0);
    }

    strategy = _strategy;

    if (address(_strategy) != address(0)) {
        if (address(assets) != nativeAddress) {
            IERC20(assets).approve(address(_strategy), 0);
            IERC20(assets).approve(address(_strategy), type(uint256).max)
                ;
        }
    }
}
```

Team Response

Fixed.

[I-02] Mutable `setAssets()`, `setCoreAddress()`, and `setMainChef()` Can Break Vault Routing and Lock User Operations

Severity

Informational Risk

Description

The vault owner can call `setAssets(IERC20 _assets)`, `setCoreAddress(address _coreAddress)`, and `setMainChef(address _mainChef)` at any time, even after users have active deposits.

Vault transfer and accounting paths depend on two mutable values:

- `assets` (the tracked token)
- `nativeAddress` (the sentinel that selects native-vs-ERC20 mode)

Vault access and transfer source paths also depend on:

- `mainChef` (the only caller allowed to execute deposit/withdraw, and the ERC20 pull source)

Because mode checks use `if (address(assets) == nativeAddress)`, mutating either value post-deposit can push the vault into the wrong execution branch or wrong token address. After a bad update, the vault can read balances and attempt transfers against incompatible addresses.

Using `address(0)` as a sentinel can be a valid design choice. The issue here is not the sentinel value itself, but allowing these core routing parameters to change while funds are active.

This is an admin-triggered configuration risk (not a permissionless exploit), but it can still disrupt user operations until corrected.

Location of Affected Code

File: vaults/SpringXVault.sol#L116-L134

```
function setAssets(IERC20 _assets) external onlyOwner {
    assets = _assets;
    emit SetAssets(address(_assets));
}

function setCoreAddress(address _coreAddress) external onlyOwner {
    nativeAddress = _coreAddress;
    emit SetCoreAddress(_coreAddress);
}

function setMainChef(address _mainChef) external onlyOwner {
    mainChef = _mainChef;
    emit SetMainChef(address(_mainChef));
}

function depositTokenToVault(address _userAddr, uint256 _amount) public
   payable nonReentrant returns (uint256){
```

```
    require(msg.sender == mainChef, "!mainChef");
    // code
    _depositAmount = _deposit(_userAddr, mainChef, _amount);
}

function _deposit(address _userAddr, address _mainChef, uint256 _amount)
   private returns (uint256){
    if (address(assets) == nativeAddress) {
        // native-mode path
    } else {
        // erc20-mode path
    }
    uint256 _poolBalance = balance();
    TransferTokenHelper.safeTokenTransferFrom(address(assets), _mainChef,
        address(this), _amount);
    uint256 _afterPoolBalance = balance();
    uint256 _depositAmount = _afterPoolBalance - _poolBalance;
    // code
}

function withdrawTokenFromVault(address _userAddr, uint256 _amount)
   public nonReentrant returns (uint256){
    require(msg.sender == mainChef, "!mainChef");
    if (address(assets) == nativeAddress) {
        // native-mode path
    } else {
        TransferTokenHelper.safeTokenTransfer(address(assets), _userAddr,
            _amount);
    }
}

function balance() public view returns (uint256) {
    if (address(assets) == nativeAddress) {
        // native-mode accounting
    } else {
        // erc20-mode accounting
    }
}
```

Impact

A privileged change to `assets`, `nativeAddress`, or `mainChef` while deposits are active can misroute vault mode/token logic or break the authorized caller/token-pull path, causing deposit/withdraw failures and misleading accounting until configuration is restored.

Proof of Concept

Scenario A — ERC20 vault misrouting via `setAssets`

1. Deploy/configure a vault for token A and connect it to the farm.
2. User deposits token A successfully.

3. Owner calls `setAssets(tokenB)` where token B is incompatible with the existing pool setup.
4. User calls `withdraw()` through the farm path.
5. Vault withdrawal path uses `safeTokenTransfer(address(assets), user, amount)` with token B and reverts.
6. User calls `deposit()` through the farm path.
7. Vault deposit path uses `safeTokenTransferFrom(address(assets), mainChef, vault, amount)` with token B and reverts.

Scenario B — Native vault mode flip via `setCoreAddress()`

1. Configure a native vault where `address(assets) == nativeAddress`.
2. User deposits native assets successfully.
3. Owner calls `setCoreAddress(newSentinel)`.
4. The mode check no longer matches and the vault uses the ERC20 branch.
5. Withdrawal attempts invoke ERC20 transfer logic using a non-ERC20 sentinel address and revert.

Scenario C — Farm/Vault lockout via `setMainChef`

1. Vault is live and integrated with the farm contract `mainChefA`.
2. Users have active balances in the vault.
3. Owner calls `setMainChef(mainChefB)` while the farm still routes calls from `mainChefA`.
4. Deposits and withdrawals from `mainChefA` now revert at `require(msg.sender == mainChef, "!mainChef")`.
5. In ERC20 mode, even if `mainChefB` calls deposit, token pull uses `safeTokenTransferFrom(..., mainChefB,` , so missing allowances/balances on `mainChefB` can revert deposits.

Restoring the original configuration can recover normal operation, which is why this remains QA-tier.

Recommendation

Treat both values as immutable during active operation, or gate setter calls to empty-vault states. If `address(0)` is intentionally used as a sentinel, keep that design but prevent live mode/asset mutation:

```
function setAssets(IERC20 _assets) external onlyOwner {
    require(totalAssets == 0, "cannot change asset with active deposits")
        ;
    assets = _assets;
    emit SetAssets(address(_assets));
}

function setCoreAddress(address _coreAddress) external onlyOwner {
    require(totalAssets == 0, "cannot change mode with active deposits");
    nativeAddress = _coreAddress;
    emit SetCoreAddress(_coreAddress);
}

function setMainChef(address _mainChef) external onlyOwner {
```

```
    require(totalAssets == 0, "cannot change mainChef with active
        deposits");
    mainChef = _mainChef;
    emit SetMainChef(_mainChef);
}
```

Team Response

Fixed.

[I-03] `SpringXVault` Missing Storage Gap `__gap`

Severity

Informational Risk

Description

`SpringXVault` is an upgradeable contract inheriting from `Initializable`, `OwnableUpgradeable`, and `ReentrancyGuardUpgradeable`, but does not declare a `uint256[50] private __gap` storage reserve. The sibling `SpringXPointFixedApyFarm` contract correctly includes one.

Since `SpringXVault` is currently a leaf contract (nothing inherits from it), this poses no immediate risk — new state variables in a future upgrade can simply be appended after existing slots. However, if `SpringXVault` ever becomes a base contract, the absence of a gap could cause storage collisions in derived contracts.

Location of Affected Code

File: vaults/SpringXVault.sol#L116-L134

```
contract SpringXVault is IVault, Initializable, OwnableUpgradeable,
    ReentrancyGuardUpgradeable {
    // ... 6 state variables, no __gap declared
}
```

Impact

No current impact. Best-practice gap for future upgrade safety.

Recommendation

Add a storage gap at the end of the contract's state variables:

```
uint256[50] private __gap;
```

Team Response

Fixed.

[I-04] `_syncUserArrays()` Comment vs Behavior Mismatch (Documentation Issue)

Severity

Informational Risk

Description

The current implementation initializes new user debt slots to zero when a new reward is added. Combined with the reward lifecycle, this makes already-staked users earn from reward go-live time onward.

That behavior is economically coherent for this design and was explicitly confirmed by the protocol team. The issue is that earlier inline comments and prior write-up wording described a different intent (start only at next user interaction), creating a documentation mismatch and false-positive risk during review.

Location of Affected Code

File: farm/SpringXPointFixedApyFarm.sol

```
function addPoolReward(
    uint256 poolId_,
    address token_,
    uint256 annualRate_,
    bool mintable_
) external onlyOwner {
    // settle existing rewards first
    _updatePool(poolId_);

    // new reward starts at zero integral at add-time
    rewards.push(RewardInfo({
        token: token_,
        rewardDecimals: rewardDecimals,
        annualRate: annualRate_,
        rateIntegral: 0,
        mintable: mintable_
    }));
}

function _syncUserArrays(uint256 poolId_, address user_) internal {
    uint256 count = poolRewards[poolId_].length;
    uint256[] storage debts = userRewardDebts[poolId_][user_];
    while (debts.length < count) debts.push(0);
}

function _settleUser(uint256 poolId_, address user_) internal {
    // pending based on currentIntegral - userDebt
    uint256 integralDelta = rewards[i].rateIntegral - debts[i];
}
```

Impact

Mismatch between the docs and the actual implementation.

Recommendation

Keep implementation as-is if this is intended economics, and align all comments/docs to match the actual model.

Team Response

Fixed.

[I-05] `poolUsers` `EnumerableSet` Grows Monotonically

Severity

Informational Risk

Description

Users are added to `poolUsers` on deposit, but there is no corresponding removal when a user fully exits via `withdraw()` or `emergencyWithdraw()`.

As a result, `poolUsers` behaves like an append-only set of historical participants rather than current active stakers. This is primarily a state-hygiene and data-quality issue (QA), not a direct fund-loss vulnerability.

Location of Affected Code

File: farm/SpringXPointFixedApyFarm.sol

```
function deposit(uint256 poolId_, uint256 amount_) external payable
    nonReentrant whenNotPaused {
    // code
    user.amount += actualAmount;
    pool.totalStaked += actualAmount;
    _updateAllRewardDebts(poolId_, msg.sender);

    poolUsers[poolId_].add(msg.sender);
}

function withdraw(uint256 poolId_, uint256 amount_) external nonReentrant
    whenNotPaused {
    // code
    user.amount -= amount_;
    pool.totalStaked -= amount_;
    _updateAllRewardDebts(poolId_, msg.sender);

    _distributeRewards(poolId_, msg.sender);
    pool.vault.withdrawTokenFromVault(msg.sender, amount_);
}
```

```
function emergencyWithdraw(uint256 poolId_) external nonReentrant {
    // code
    uint256 amount = user.amount;

    user.amount = 0;
    pool.totalStaked -= amount;

    // ... no poolUsers remove ...

    pool.vault.withdrawTokenFromVault(msg.sender, amount);
}
```

Impact

Stale user entries accumulate permanently per pool.

Recommendation

Remove users from `poolUsers` when their stake reaches zero.

```
function withdraw(uint256 poolId_, uint256 amount_) external nonReentrant
    whenNotPaused { // remove only on full exit
  // code
  user.amount -= amount_;
  pool.totalStaked -= amount_;
  if (user.amount == 0) {
      poolUsers[poolId_].remove(msg.sender);
  }
  // code
}

function emergencyWithdraw(uint256 poolId_) external nonReentrant { //
   always a full exit
  // code
  uint256 amount = user.amount;
  user.amount = 0;
  pool.totalStaked -= amount;
  poolUsers[poolId_].remove(msg.sender);
  // code
}
```

Team Response

Fixed.

[I-06] Deposit/Withdraw Fee-on-Transfer Handling Inconsistency

Severity

Informational Risk

Description

Token-transfer accounting is not fully symmetric across deposit and withdraw flows for fee-on-transfer style behavior.

- On deposit, `SpringXVault._deposit()` uses balance-delta accounting (actual received amount), but the Farm first pulls `amount_` from the user to the Farm and then forwards the same nominal `amount_` to the vault.
- On withdrawal, vault accounting is decremented by nominal `_amount`, and the transfer path sends nominal `_amount` without validating the actual user-received amount.

Location of Affected Code

File: farm/SpringXPointFixedApyFarm.sol#L270-L298

```
function deposit(uint256 poolId_, uint256 amount_) external payable
    nonReentrant whenNotPaused {
  // code
  // Farm deposit first hop: user -> farm uses nominal amount
  IERC20(pool.stakeToken).safeTransferFrom(msg.sender, address(this),
      amount_);
  actualAmount = pool.vault.depositTokenToVault(msg.sender, amount_);

  // Vault deposit second hop: mainChef -> vault uses balance delta (FOT-
      aware)
  uint256 _poolBalance = balance();
  TransferTokenHelper.safeTokenTransferFrom(address(assets), _mainChef,
      address(this), _amount);
  uint256 _afterPoolBalance = balance();
  uint256 _depositAmount = _afterPoolBalance - _poolBalance;

  // Vault withdraw path: accounting decremented by nominal amount, no
      actual-received check
  _userInfo.amount = _userInfo.amount - _amount;
  totalAssets = totalAssets - _amount;
  TransferTokenHelper.safeTokenTransfer(address(assets), _userAddr,
      _amount);
  // code
}
```

Impact

In consistency between the working of the deposit and withdrawal paths.

Recommendation

Either accept them fully or disregard them from both paths.

Team Response

Acknowledged.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!